Merge and Sort Intervals

Given an array of intervals [startTime, endTime], merge all overlapping intervals and return a sorted array of non-overlapping intervals.

Example

Input

```
intervals = [[1, 3], [2, 6], [8, 10], [15, 18]]
```

Output

```
[[1, 6], [8, 10], [15, 18]]
```

Explanation

```
- Step 1: Sort intervals by start time (already sorted).
- Step 2: Initialize merged list with first interval [1,3].
- Step 3: Compare [2,6] with last merged [1,3]. They overlap (2 ≤ 3), so merge into [
- Step 5: Compare [15,18] with last merged [8,10]. No overlap (15 > 10), append [15,1

Result: [[1,6],[8,10],[15,18]].
```

Input Format

The first line contains an integer denoting the number of intervals.

The second line contains an integer denoting the length of individual interval array.

Each of the next N lines contains two space-separated integers startTime and endTime

Intervals may be provided in any order; duplicates and fully contained intervals are allowed.

Example

```
4
2
1 3
2 6
8 10
15 18
```

here, 4 is the number of intervals, 2 is the length of individual interval array, followed by the intervals.

Constraints

0 <= intervals.length <= 100000

intervals[i].length == 2 for all 0 <= i < intervals.length

0 <= intervals[i][0] < intervals[i][1] <= 10^9 for all 0 <= i < intervals.length

Output Format

Return a 2D array of two space-separated integers start and end, representing the merged intervals sorted by increasing start time.

Sample Input 0

```
0
0
```

Sample Input 1

```
1
2
5 10
```

Sample Output 1

```
5 10
```

Language: Python 3

```
#!/bin/python3⋯

#
# Complete the 'mergeHighDefinitionIntervals' function below.
#
# The function is expected to return a 2D_INTEGER_ARRAY.
# The function accepts 2D_INTEGER_ARRAY intervals as parameter.
#

def mergeHighDefinitionIntervals(intervals):
    # Write your code here

if __name__ == '__main__':⋯
```

Ln 10, Col 1 · Autocomplete · Spaces: 4 · Mode: Normal

Test Results

Run Code Submit Code

AI Tutor